Learning to Escape Police with Double DQN

Askat Aubakirov and Alnur Shldenov
Group BDA2403 | Deep Learning Endterm Project | 9 November 2026

1 Objective

We study whether a neural network agent can survive a police chase and reach a safehouse in our own top-down driving game, Night Shift. The car has inertia, drifting, collisions and limited nitro. The safehouse opens after 45 simulated seconds. A successful episode ends when the player reaches its 75-pixel radius after opening. Capture or destruction ends the episode with failure; an external limit truncates an unfinished chase after 180 seconds.

This is reinforcement learning because the agent learns from action-dependent rewards while interacting with the simulator. There are no labelled steering examples. An action that is useful immediately, such as avoiding a collision, can also change the chance of escaping later. Our objective is a policy with high escape success and high cumulative game reward. We compare two network sizes, then test adaptation from the original game V1 to a stricter capture rule in V2.

In the MDP description (S, A, P, R, gamma), S is the full game state, A is the control set, P is the simulator transition rule, R is the reward below, and gamma = 0.995. The policy acts on a partial observation of S.

The simulator state contains the car, police, map, timers and routing memory. The agent receives 60 normalized numerical observations, clipped to [-1, 1]: 23 vehicle and task features, 21 police features and 16 obstacle-clearance rays. Features include velocity, heading, health, nitro, capture progress, relative goal direction and time until opening. Police routing memory is hidden, so the observation is only an approximation to a Markov state. There are 15 discrete control combinations, including acceleration, steering, braking/reverse, drifting and nitro. Each decision advances 0.1 seconds through 12 physics steps.

The original reward gives +0.05 per second before opening and -0.05 per second afterwards, +0.002 per pixel of signed goal progress after opening, -0.1 per health point lost, -0.5 times positive capture-meter growth, +20 for escape and -10 for failure. The notebook lists the individual terms. Evaluation always uses this original reward; extra training shaping is excluded.

2 Related Work and Connection to the Lecture

Lecture 9 introduces the agent-observation-action-reward loop, discounted return, Q-learning, epsilon-greedy exploration and the Gym API (Zholtayev, n.d.). We keep this structure and replace the Q-table with a neural network because car positions and velocities are continuous. Rendering is separate: headless training uses the same game physics, and the rotating camera is not part of the numerical input.

DQN combines Q-learning with a neural network, experience replay and, in its later formulation, a target network (Mnih et al., 2013, 2015). We use Double DQN: the online network selects the next action and the target network evaluates it (van Hasselt et al., 2015). This addresses one source of overestimated Q-values; it does not guarantee stable learning. Potential-based shaping supplies additional goal feedback during training (Ng et al., 1999). Gymnasium and the PyTorch DQN tutorial informed the API and implementation structure; our environment, action set, reward design and experiments are specific to this project.

3 Technical Approach and Implementation

Both agents use a multilayer perceptron with ReLU hidden layers and 15 linear outputs. Each output estimates the discounted return of one action. B has hidden layers [64, 64] and 9,039 parameters. C has [128, 128, 64] and 33,551 parameters. In the V1 comparison, hidden-layer sizes are the only configuration difference between B and C. Both runs start from scratch with seed 42. The overnight target was initially 400,000 steps; after interruption we use their common saved 300,000-step boundary for the fixed-budget comparison.

A transition stores observation, action, reward, next observation and the termination flag. Epsilon-greedy exploration chooses a random action with probability epsilon; otherwise it uses the largest predicted Q-value. A replay buffer reduces the dependence between consecutive samples. After warm-up, Adam updates the online network on a random batch every four environment steps. The target network receives a hard copy of the online weights every 1,000 environment steps. We use Smooth L1 (Huber) loss and clip the gradient norm to 10.

```
G_t = sum over k of gamma^k * r_(t+k+1)
```

```
a* = argmax_a Q_online(o_next, a)
y = r + gamma * (1 - terminated) * Q_target(o_next, a*)
loss = mean Huber(Q_online(o, a) - y)
```

The target is calculated without gradients. A true terminal state removes the bootstrap term. A time-limit truncation resets the episode but still bootstraps, following Gymnasium semantics. The selected demonstration policy is frozen; the notebook performs its teaching update on a separate network copy.

For goal shaping we set Phi(s) = -distance_to_safehouse / world_diagonal and add 8 * [gamma * Phi(s_next) - Phi(s)] to the training reward. Phi is zero at a true terminal state and is retained at a time-limit truncation. This feedback is available before the exit opens, so the agent can learn to approach the future goal. The environment already contains a heuristic signed-progress term; we therefore do not claim that the complete reward design guarantees an unchanged optimal policy.

Setting	B and C
Discount and optimizer	gamma 0.995; Adam; learning rate 0.0001
Replay and update	50,000 transitions; batch 64; warm-up 2,000 steps
Update frequency	Online every 4 steps; target copy every 1,000 steps
Exploration in V1	Epsilon 1.0 to 0.1 over 100,000 steps
Exploration in V2	Epsilon 0.4 to 0.05 over 200,000 additional steps
Training budget	V1: common 300,000 steps; V2: 400,000 additional steps each
Training reward	Original game reward plus potential shaping with scale 8
Evaluation	Greedy actions, original reward, no optimizer updates

V2 changes only the capture-release condition, not police speed or the reward coefficients. Capture grows when a police car is within 72 pixels and the player is slower than 65 pixels/second. In V1, short speed bursts could make the meter fall even near police. V2 holds the meter while any police car is within 120 pixels and releases it only when all police cars are farther away. V2 training starts from the selected V1 weights, with a fresh optimizer, target copy and replay buffer. Half of training starts are exact; half use offsets up to 30 pixels per axis and 10 degrees. B starts from V1 step 250k; C starts from V1 step 300k.

4 Results and Discussion

4.1 Network Size and Results in V1

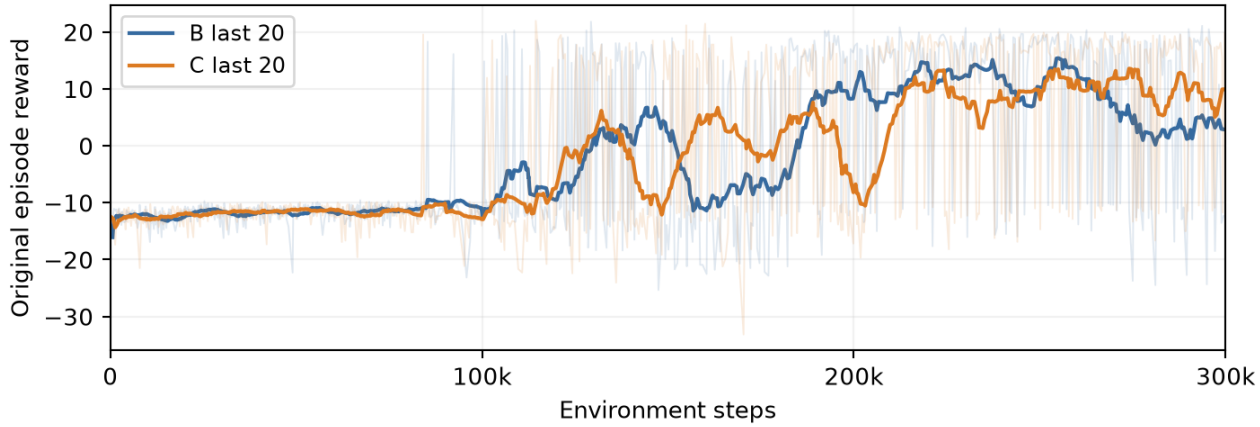


Figure 1. Original episode reward versus environment steps in V1. Faint lines show individual episodes; solid lines show the last-20-episode mean. Both configurations use the same 300k comparison boundary. These exploratory training curves are not test success rates.

At the matched 300k boundary, B won 9/20 validation starts and C won 16/20. Their mean original rewards were -2.52 ± 16.77 and 12.05 ± 12.00. This supports using the larger network in this run, but one seed does not establish a general rule that deeper networks are better. B performed better at 250k than at 300k (14/20 versus 9/20), showing why the last checkpoint is not automatically the best.

We separated 20 validation starts from 40 held-out changed starts. The validation score selected C300k. A supplementary deployment criterion requiring success on the ordinary fixed start selected B250k before its held-out test. The held-out test cases were the same for both candidates. The selected snapshots differ in step count, although both training runs reached the common 300k boundary.

V1 held-out policy	Wins / 40	Reward mean ± SD	Mean winning time
B 250k	33	12.72 ± 11.54	57.59 s
C 300k	36	15.18 ± 9.15	46.97 s

C300k escaped in 36/40 changed starts, but failed on the exact original start after 21.55 seconds. B250k won that fixed start. This difference matters: repeated greedy tests with the same start can reproduce the same trajectory, and changing a seed alone did not create a different map or starting position. We therefore report the fixed case separately rather than treating its ten repeats as ten independent scenarios.

The V1 videos also exposed an environment weakness. Some behaviour relied on small acceleration bursts to reduce capture while police stayed close. This is an unintended consequence of the rules, not evidence that the agent understood or deliberately cheated. We preserved V1 and changed the release condition in V2 so that escape from immediate police pressure requires physical distance. Raw V1 and V2 scores use different test protocols and dynamics, so their percentages are not a direct before/after causal comparison.

4.2 Adaptation and Model Choice in V2

For V2, each candidate received 400k additional steps. Every 50k steps we checked five fixed diagnostic starts. Selection prioritized an ordinary-start win, then diagnostic win rate, mean reward and later step; step 0 was eligible. C400k was selected with 4/5 diagnostic wins. For B, the original policy remained best by this rule with 5/5. The uploaded B400k policy is therefore reported as the latest policy, not as the best retained policy.

Policy in V2	Changed wins / 40	Reward mean ± SD	Fixed wins / 10
B original / selected	32	9.83 ± 12.02	10
B latest +400k	13	-1.60 ± 13.73	0
C original	29	8.60 ± 14.14	0
C selected +400k	38	17.35 ± 6.85	10

Table 2. Matched V2 evaluation on the same computer: 40 distinct changed starts and ten repeats of the ordinary start. All policies act greedily and receive original reward. SD is population episode-reward variability, not uncertainty across training seeds. The changed starts were excluded from checkpoint selection.

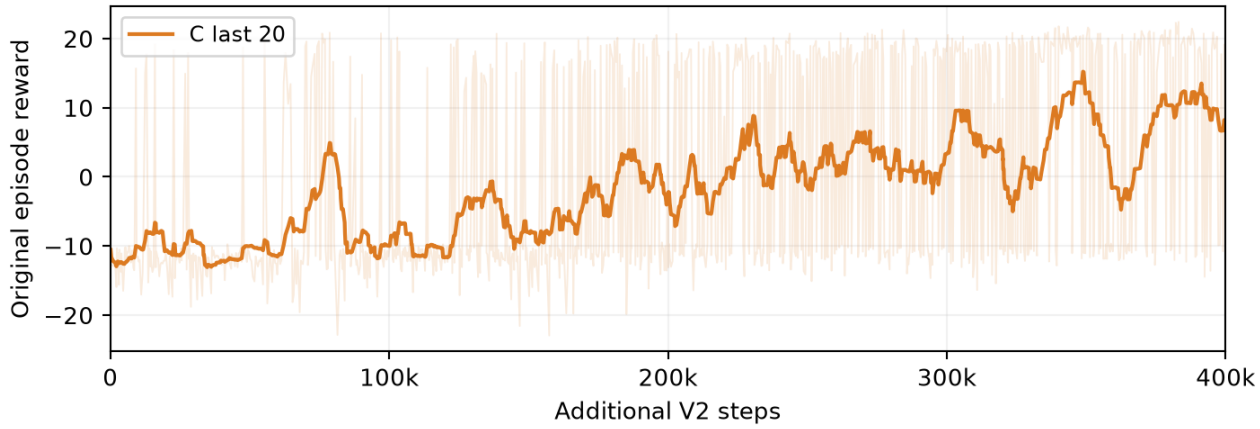


Figure 2. C during its 400k-step V2 adaptation, using original episode reward. The notebook also shows B/C diagnostic curves. B episode-level V2 training history has not been transferred, so we do not invent a corresponding reward curve.

C improved from 29/40 to 38/40 changed-start wins and solved the ordinary start in 48.33 seconds. B decreased from 32/40 to 13/40 and its latest policy failed the ordinary start in 6.77 seconds. The imported B weights reproduced all five reported 400k diagnostics, so this observed decline is not explained by a simple file-transfer error. Training was unstable; these data alone cannot distinguish overfitting, forgetting and other causes of value-learning instability.

We select C400k for the final demonstration. Its mean reward on changed starts is 17.35 ± 6.85 and its mean time among successful episodes is 56.79 seconds. This is stronger evidence than a single successful video. However, the study uses one training seed and one map with small start perturbations. V2 also starts from unequal V1 budgets (B250k/C300k), so the adaptation comparison is not an architecture-only experiment. B ran on a reported RTX 4080 SUPER and C on an RTX 4060 Laptop; their training times are not an efficiency comparison.

5 Conclusion and Future Work

A self-implemented Double DQN agent learned to escape our custom driving game. C achieved 95% success on the 40 changed V2 starts, while B illustrates that further training can reduce performance. Next we would repeat both configurations across several training seeds and train on multiple maps; a recurrent policy could also address hidden police-routing state.

References

Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., & Riedmiller, M. (2013). Playing Atari with deep reinforcement learning. arXiv:1312.5602.
https://arxiv.org/abs/1312.5602

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. Nature, 518, 529-533.
https://doi.org/10.1038/nature14236

van Hasselt, H., Guez, A., & Silver, D. (2015). Deep reinforcement learning with Double Q-learning. arXiv:1509.06461.
https://arxiv.org/abs/1509.06461

Ng, A. Y., Harada, D., & Russell, S. (1999). Policy invariance under reward transformations: Theory and application to reward shaping. Proceedings of ICML, 278-287.
https://ai.stanford.edu/~ang/papers/shaping-icml99.pdf

Farama Foundation. (n.d.). Gymnasium environment API. Retrieved November 9, 2026.
https://gymnasium.farama.org/api/env/

Paszke, A., & Towers, M. (n.d.). Reinforcement learning (DQN) tutorial. PyTorch Tutorials. Retrieved November 9, 2026.
https://docs.pytorch.org/tutorials/intermediate/reinforcement_q_learning.html

Zholtayev, D. (n.d.). AI in robotics [Lecture 9 slides]. Deep Learning course. Course-provided PDF; see especially slides 13-20, 38-43 and 47-51.

Project Materials

Submission and reproduction. The notebook includes visible learning steps, saved tables and curves, and a short frozen-policy replay. Full training is optional. The attached project_assets.zip contains the custom environment, weights and evidence; Night_Shift_Demo.mp4 shows the recorded gameplay. No external dataset or paid API is required. B full replay/optimizer history is not included; its transferred weights and diagnostic summary are retained.

Aubakirov, A., & Shldenov, A. (2026). Night Shift: custom Gymnasium game, Double DQN experiments, model weights and recorded evaluation. Project repository and final submission folder:

https://github.com/AskatQwerty/Reinforcement/tree/codex/final-submission/submission

The project notebook explains the additions to the standard DQN pattern: numerical vehicle observations, 15 driving actions, the custom capture mechanic, goal-potential shaping, the network-size comparison and the V1-to-V2 adaptation experiment. References provide the methods and API background; the reported numerical results come from the saved project runs.